CS 220 — Data Structures & Algorithms: Syllabus

Course CS 220 introduces the fundamental data structures used throughout computer science — arrays, linked lists, stacks, queues, hash tables, trees, and graphs — along with the algorithmic techniques for searching, sorting, and analyzing them.

Prerequisites: CS 110 (Introduction to Programming) or equivalent experience in a modern programming language. Comfort with recursion and basic time-complexity reasoning is expected by mid-semester.

Textbook: "Introduction to Algorithms" by Cormen, Leiserson, Rivest, and Stein (CLRS), plus the freely available course lecture notes distributed each week on the course portal.

Grading: homework assignments 30%, one midterm 20%, a final exam 25%, and a semester-long programming project worth 25%. The project requires implementing and benchmarking a balanced search tree against a hash table.

The midterm covers arrays, linked lists, stacks, queues, and complexity analysis. The final exam is cumulative but emphasizes trees, graphs, and algorithm design patterns such as divide and conquer and dynamic programming.

Academic honesty: all submitted work must be your own. Reusing code from online repositories without citation is a violation and results in a zero for the assignment plus a report to the university conduct board.

CS 220 — Weekly Schedule (Weeks 1–7)

Week 1: Mathematical foundations — growth of functions, Big-O, Big-Theta, and Big-Omega notation. Week 2: Arrays and dynamic arrays, amortized cost of resizing. Week 3: Singly and doubly linked lists; comparing contiguous and linked storage.

Week 4: Stacks and queues, including applications in expression evaluation and breadth-first search. Week 5: Hash tables — hash functions, collision resolution by chaining and open addressing, load factor and expected cost.

Week 6: Binary search trees, tree traversals, and the BST property. Week 7: Balanced trees — AVL rotations and red-black tree invariants. Homework 3 asks you to implement an AVL tree from scratch in C++ or Python.

Weeks 8–10: Heaps and priority queues, then heapsort. Weeks 11–12: Graph representations — adjacency lists versus adjacency matrices — and graph traversals BFS and DFS.

Weeks 13–14: Shortest paths (Dijkstra and Bellman-Ford) and minimum spanning trees (Kruskal and Prim). The final project is due in week 15, before the exam week.